

07 — Special Data Types in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Range Types](#)
 3. [Network Address Types](#)
 4. [Geometric Types](#)
 5. [Full-Text Search Types — tsvector & tsquery](#)
 6. [XML Type](#)
 7. [Other Notable Types](#)
 8. [SQL Examples](#)
 9. [Common Mistakes](#)
 10. [Best Practices](#)
 11. [Performance Considerations](#)
 12. [Interview Questions & Answers](#)
 13. [Exercises with Solutions](#)
 14. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Use range types for booking, scheduling, and validity-period problems
 - Store and query IP addresses, MAC addresses, and CIDRs natively
 - Use `tsvector` and `tsquery` for full-text search
 - Understand when PostgreSQL's geometric types are appropriate
 - Use the XML type with XPath queries
-

Range Types

Built-in range types

```
Range Types
├─ int4range    – integer range
├─ int8range    – bigint range
├─ numrange     – numeric range
├─ tsrange      – timestamp (no TZ) range
├─ tstzrange    – timestampz range
└─ daterange    – date range
```

Range syntax

```
Bounds notation:
[a, b]  – inclusive on both sides (a ≤ x ≤ b)
(a, b)  – exclusive on both sides (a < x < b)
[a, b)  – inclusive lower, exclusive upper (a ≤ x < b) ← most common for dates
(a, b]  – exclusive lower, inclusive upper (a < x ≤ b)
empty   – empty range (no values)
```

```

-- Creating ranges
SELECT '[1, 10]':int4range;      -- [1,11) - normalized to half-open
SELECT '[1, 10)':int4range;      -- [1,10)
SELECT '(1, 10]':int4range;      -- [2,11) - normalized
SELECT 'empty':int4range;        -- empty

SELECT '[2024-01-01, 2024-12-31]':daterange; -- [2024-01-01,2025-01-01)
SELECT '[2024-01-01, 2024-12-31)':daterange; -- [2024-01-01,2024-12-31)

-- Constructor functions
SELECT int4range(1, 10);          -- [1,10) - lower inclusive, upper exclusive by default
SELECT int4range(1, 10, '[');    -- [1,10]
SELECT daterange('2024-01-01', '2024-12-31', '[');

-- Access bounds
SELECT lower('[1,10)':int4range); -- 1
SELECT upper('[1,10)':int4range); -- 10
SELECT lower_inc('[1,10)':int4range); -- TRUE (lower is inclusive)
SELECT upper_inc('[1,10)':int4range); -- FALSE (upper is exclusive)
SELECT isempty('[1,10)':int4range); -- FALSE

```

Range operators

```

-- Containment
SELECT '[1,10)':int4range @> 5;      -- TRUE (5 is in range)
SELECT '[1,10)':int4range @> '[2,5)':int4range; -- TRUE (sub-range contained)

-- Overlap
SELECT '[1,5)':int4range && '[3,8)':int4range; -- TRUE (overlap 3-5)
SELECT '[1,3)':int4range && '[5,8)':int4range; -- FALSE (no overlap)

-- Strictly left/right
SELECT '[1,5)':int4range << '[6,10)':int4range; -- TRUE (left is strictly less)
SELECT '[6,10)':int4range >> '[1,5)':int4range; -- TRUE (right is strictly greater)

-- Adjacent
SELECT '[1,5)':int4range -|- '[5,10)':int4range; -- TRUE (adjacent, no gap)

-- Union, intersection, difference
SELECT '[1,5)':int4range + '[3,8)':int4range;    -- [1,8)
SELECT '[1,5)':int4range * '[3,8)':int4range;    -- [3,5)
SELECT '[1,8)':int4range - '[3,5)':int4range;    -- ERROR (would split into two)

```

Custom range types

```

-- Create a range type for any existing type
CREATE TYPE floatrange AS RANGE (

```

```

        subtype = float8,
        subtype_diff = float8mi
    );

SELECT '[1.5, 4.5]':floatrange;
SELECT '[1.5, 4.5]':floatrange @> 3.0; -- TRUE

```

EXCLUSION constraint using ranges

```

-- Prevent booking overlaps
CREATE TABLE room_bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    INTEGER NOT NULL,
    guest_name TEXT    NOT NULL,
    stay       daterange NOT NULL,
    CONSTRAINT no_overlapping_bookings
        EXCLUDE USING GIST (room_id WITH =, stay WITH &&)
);

-- This will fail if room 101 is already booked for overlapping dates
INSERT INTO room_bookings (room_id, guest_name, stay) VALUES
(101, 'Alice', '[2024-07-01, 2024-07-07]');
-- Succeeds

INSERT INTO room_bookings (room_id, guest_name, stay) VALUES
(101, 'Bob', '[2024-07-05, 2024-07-10]');
-- ERROR: conflicting key value violates exclusion constraint

```

Network Address Types

inet — IP address + optional subnet

```

SELECT '192.168.1.1':inet;           -- IPv4
SELECT '192.168.1.0/24':inet;        -- IPv4 with subnet
SELECT '::1':inet;                   -- IPv6 loopback
SELECT '2001:db8::/32':inet;         -- IPv6 with subnet

-- Functions
SELECT host('192.168.1.1/24':inet);  -- '192.168.1.1' (just the host)
SELECT masklen('192.168.1.1/24':inet); -- 24 (subnet mask length)
SELECT netmask('192.168.1.1/24':inet); -- '255.255.255.0'
SELECT network('192.168.1.1/24':inet); -- '192.168.1.0/24' (network address)
SELECT broadcast('192.168.1.0/24':inet); -- '192.168.1.255/24'
SELECT abbrev('2001:db8::1/128':inet); -- abbreviated IPv6

-- Operators
SELECT '192.168.1.5':inet << '192.168.1.0/24':inet; -- TRUE (contained in
subnet)
SELECT '192.168.1.0/24':inet >> '192.168.1.5':inet; -- TRUE (contains IP)

```

```
SELECT '192.168.1.0/24'::INET >= '192.168.1.0/24'::INET; -- TRUE (contains or equals)

-- Arithmetic
SELECT '192.168.1.0'::INET + 5;    -- '192.168.1.5'
SELECT '192.168.1.10'::INET - '192.168.1.0'::INET;  -- 10
```

cidr — network address only (no host bits)

```
SELECT '192.168.1.0/24'::CIDR;    -- valid
SELECT '192.168.1.5/24'::CIDR;    -- ERROR: host bits set (must be .0 for /24)
-- Use CIDR when storing network addresses; use INET when storing host addresses
```

macaddr — MAC address

```
SELECT '08:00:2b:01:02:03'::MACADDR;
SELECT 'macaddr'::TEXT;    -- canonical format: 08:00:2b:01:02:03
SELECT trunc('12:34:56:78:90:ab'::MACADDR);  -- 12:34:56:00:00:00 (zero last 3 bytes)

-- macaddr8 for EUI-64 (8-byte MAC, e.g., modern network interfaces)
SELECT '08:00:2b:01:02:03:04:05'::MACADDR8;
```

Network type comparison

Type	Storage	Description
inet	7 bytes	IPv4 or IPv6 with optional subnet
cidr	7 bytes	network address only (strict: host bits must be 0)
macaddr	6 bytes	IEEE 802 MAC address (EUI-48)
macaddr8	8 bytes	IEEE 802 MAC address (EUI-64)

Geometric Types

```
Geometric Types
├─ point    - (x, y) coordinate
├─ line     - infinite line
├─ lseg     - line segment
├─ box      - rectangular box
├─ path     - open or closed path
├─ polygon  - closed polygon
└─ circle   - center + radius
```

```
-- point
SELECT '(1.5, 2.3)'::POINT;
SELECT point(1.5, 2.3);
```

```
-- box
SELECT '((0,0),(10,10))'::BOX;

-- circle
SELECT '<(0,0),5>'::CIRCLE; -- center (0,0), radius 5

-- Functions & operators
SELECT '(1,1)'::POINT <-> '(4,5)'::POINT; -- Euclidean distance = 5.0
SELECT '(0,0)'::POINT <@ '((-1,-1),(1,1))'::BOX; -- TRUE (point in box)
SELECT '((0,0),(2,2))'::BOX && '((1,1),(3,3))'::BOX; -- TRUE (overlap)

-- Distance operators
SELECT |/ (3^2 + 4^2); -- 5 (Pythagorean distance – another way)
```

Note: For serious geospatial work, use the **PostGIS** extension which provides a far richer set of spatial types, functions, and indexes (GIST, BRIN). The built-in geometric types are useful for simple 2D calculations.

Full-Text Search Types — tsvector & tsquery

tsvector — document representation

A **tsvector** is a sorted list of **lexemes** (normalized word stems) with optional position and weight information.

```
Input text: "The quick brown fox jumps over the lazy dog"
tsvector:  'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2
           ^- stopwords removed ('the', 'over')
           ^- words stemmed ('jumps' → 'jump', 'lazy' → 'lazi')
           ^- positions stored (':3' means 3rd word)
```

```
-- Create tsvectors
SELECT to_tsvector('english', 'The quick brown fox jumps over the lazy dog');
-- 'brown':3 'dog':9 'fox':4 'jump':5 'lazi':8 'quick':2

-- Weights (A=highest, D=lowest)
SELECT
    setweight(to_tsvector('english', 'PostgreSQL Arrays'), 'A') ||
    setweight(to_tsvector('english', 'Learn about array types in PostgreSQL'), 'B')
AS weighted_vector;
```

tsquery — search query

```
-- Boolean operators: & (AND), | (OR), ! (NOT), <-> (followed by)
SELECT to_tsquery('english', 'quick & fox');
SELECT to_tsquery('english', 'quick | slow');
SELECT to_tsquery('english', 'fox & !rabbit');
SELECT to_tsquery('english', 'quick <-> fox'); -- "quick" immediately followed by "fox"
SELECT to_tsquery('english', 'quick <2> fox'); -- "quick" within 2 words of "fox"
```

```

-- plainto_tsquery: interprets plain text, no special chars needed
SELECT plainto_tsquery('english', 'quick brown fox');
-- 'quick' & 'brown' & 'fox'

-- phraseto_tsquery: phrase search
SELECT phraseto_tsquery('english', 'quick brown fox');
-- 'quick' <-> 'brown' <-> 'fox'

-- websearch_to_tsquery: web-search style (Google-like)
SELECT websearch_to_tsquery('english', 'quick fox -rabbit "brown fox"');

```

Full-text search in practice

```

-- Table with generated tsvector column
CREATE TABLE articles (
  article_id  BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title       TEXT    NOT NULL,
  body        TEXT    NOT NULL,
  search_vec  TSVECTOR GENERATED ALWAYS AS (
    setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', body), 'B')
  ) STORED
);

-- GIN index on the tsvector column
CREATE INDEX idx_articles_fts ON articles USING GIN (search_vec);

-- Full-text search
SELECT
  article_id,
  title,
  ts_rank(search_vec, query) AS relevance
FROM articles,
  websearch_to_tsquery('english', 'postgresql arrays index') AS query
WHERE search_vec @@ query
ORDER BY relevance DESC;

-- Headline: highlight matching terms
SELECT
  title,
  ts_headline('english', body, query,
    'MaxFragments=3, MaxWords=30, MinWords=10, StartSel=<mark>, StopSel=</mark>'
  ) AS excerpt
FROM articles,
  websearch_to_tsquery('english', 'postgresql') AS query
WHERE search_vec @@ query;

-- Match operator: @@
SELECT 'fat & cat'::TSQUERY @@ 'the fat cat sat'::TSVECTOR; -- TRUE
SELECT 'fat & rat'::TSQUERY @@ 'the fat cat sat'::TSVECTOR; -- FALSE

```

```
-- Rank functions
SELECT ts_rank(to_tsvector('quick brown fox'), to_tsquery('fox'));      --
normalized rank
SELECT ts_rank_cd(to_tsvector('quick brown fox'), to_tsquery('fox'));  -- cover
density rank
```

XML Type

```
-- Store XML data
CREATE TABLE product_specs (
    spec_id      SERIAL PRIMARY KEY,
    product_id   INTEGER NOT NULL,
    spec_xml     XML NOT NULL
);

INSERT INTO product_specs (product_id, spec_xml) VALUES (
    1,
    '<product>
      <name>Laptop</name>
      <specs>
        <cpu>Intel i7</cpu>
        <ram unit="GB">16</ram>
      </specs>
    </product>'
);

-- XPath queries
SELECT xpath('/product/name/text()', spec_xml) FROM product_specs;
-- {Laptop}

SELECT xpath('/product/specs/ram/text()', spec_xml) FROM product_specs;
-- {16}

SELECT xpath('/product/specs/ram/@unit', spec_xml) FROM product_specs;
-- {GB}

-- xmltable: convert XML to rows
SELECT *
FROM xmltable(
    '/root/product'
    PASSING '<root><product><id>1</id><name>Laptop</name></product></root> '::XML
    COLUMNS
        product_id INTEGER PATH 'id',
        product_name TEXT PATH 'name'
);

-- Generate XML from relational data
SELECT xmlelement(name product,
    xmlelement(name id, product_id),
```

```
    xmlelement(name name, name)
) FROM products;

SELECT table_to_xml('products', true, false, ''); -- whole table to XML
```

Other Notable Types

pg_lsn — Log Sequence Number

```
SELECT pg_current_wal_lsn();           -- current WAL LSN
SELECT '0/16B4C28'::PG_LSN;
SELECT '0/16B4C28'::PG_LSN - '0/16B3000'::PG_LSN; -- bytes between LSNs
```

txid_snapshot — transaction snapshot

```
SELECT txid_current_snapshot(); -- current snapshot for MVCC
```

oid — Object Identifier

```
-- Used internally by PostgreSQL
SELECT oid, relname FROM pg_class LIMIT 5;
SELECT 'pg_class'::REGCLASS::OID; -- get OID of system table
SELECT 16384::REGCLASS;           -- lookup table by OID
```

citext — case-insensitive text

```
CREATE EXTENSION IF NOT EXISTS citext;
CREATE TABLE emails (email CITEXT UNIQUE);
INSERT INTO emails VALUES ('User@Example.COM');
SELECT * FROM emails WHERE email = 'user@example.com'; -- matches!
```

SQL Examples

Complete booking system with range types

```
CREATE TABLE venues (
    venue_id SERIAL PRIMARY KEY,
    name TEXT NOT NULL,
    address INET, -- IP for virtual venues
    capacity SMALLINT NOT NULL
);

CREATE TABLE bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    venue_id INTEGER NOT NULL REFERENCES venues(venue_id),
```



```

    event_name TEXT NOT NULL,
    booked_by TEXT NOT NULL,
    time_range TSTZRANGE NOT NULL,
    CONSTRAINT no_double_booking
        EXCLUDE USING GIST (venue_id WITH =, time_range WITH &&)
);

CREATE INDEX idx_bookings_venue ON bookings (venue_id);
CREATE INDEX idx_bookings_time ON bookings USING GIST (time_range);

-- Find available slots
SELECT
    b1.time_range,
    upper(b1.time_range) AS slot_start,
    lower(b2.time_range) AS slot_end
FROM bookings b1
JOIN bookings b2 ON b2.venue_id = b1.venue_id
    AND lower(b2.time_range) > upper(b1.time_range)
WHERE b1.venue_id = 1
    AND NOT EXISTS (
        SELECT 1 FROM bookings b3
        WHERE b3.venue_id = b1.venue_id
            AND b3.time_range && tstzrange(upper(b1.time_range), lower(b2.time_range))
    );

-- Bookings overlapping "now"
SELECT * FROM bookings WHERE time_range @> now();

-- Bookings in next 7 days
SELECT * FROM bookings
WHERE time_range && tstzrange(now(), now() + INTERVAL '7 days');
```

Network monitoring system

```

CREATE TABLE network_assets (
    asset_id SERIAL PRIMARY KEY,
    hostname TEXT NOT NULL,
    ip_address INET NOT NULL UNIQUE,
    mac_address MACADDR,
    subnet CIDR NOT NULL,
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    last_seen TIMESTAMPTZ
);

CREATE INDEX idx_assets_subnet ON network_assets USING GIST (ip_address inet_ops);

-- Find all assets in a specific subnet
SELECT hostname, ip_address
FROM network_assets
WHERE ip_address << '192.168.1.0/24'::INET;
```

```
-- Find assets by MAC vendor prefix (first 3 bytes)
SELECT hostname, mac_address
FROM network_assets
WHERE trunc(mac_address) = '08:00:2b:00:00:00'::MACADDR;
```

Common Mistakes

1. Not using GIST index for range overlap queries

```
-- SLOW without index
SELECT * FROM bookings WHERE time_range && tstzrange(now(), now() + INTERVAL
'1 week');
-- Add:
CREATE INDEX ON bookings USING GIST (time_range);
```

2. Using TEXT for IP addresses

```
-- BAD: no IP validation, inefficient storage, no subnet math
ip_address TEXT

-- GOOD: validated, 7 bytes, subnet operators available
ip_address INET NOT NULL
```

3. Not setting the text search configuration

```
-- BAD: uses default config (may be 'simple' or wrong language)
to_tsvector(body)

-- GOOD: explicit language
to_tsvector('english', body)
```

4. **Forgetting stopwords in full-text search** — Common words like "the", "a", "is" are removed by the text search parser. A query for "the" will return no results. This is expected behavior.

5. **Using range types without exclusion constraints** — The main value of range types in booking systems is the EXCLUDE USING GIST constraint. Without it, you get no overlap protection.

Best Practices

1. **Use range types** for any "validity period", "date range", or "booking" concept — they come with built-in overlap detection.
 2. **Use INET** for IP addresses — never TEXT. INET validates format and enables subnet math.
 3. **Use TSVECTOR generated columns** for full-text search — pre-compute at write time, not query time.
 4. **Always specify the language** in `to_tsvector()` and `to_tsquery()`.
 5. **Use `websearch_to_tsquery()`** for user-facing search — it handles mistakes gracefully.
 6. **Use GIST indexes** for range types and geometric types; GIN for tsvector.
-

Performance Considerations

```
-- Range type index: GIST
CREATE INDEX idx_bookings_range ON bookings USING GIST (time_range);
-- Supports: &&, @>, <@, <<, >>, -|- , &<, &>, adjacent operators

-- Full-text search: GIN vs GIST
-- GIN: faster queries, slower updates
-- GIST: slower queries, faster updates, supports proximity
CREATE INDEX idx_fts_gin ON articles USING GIN (search_vec); -- recommended
CREATE INDEX idx_fts_gist ON articles USING GIST (search_vec); -- high write
throughput

-- Network types: GIST for subnet containment
CREATE INDEX idx_ip_gist ON network_assets USING GIST (ip_address inet_ops);
```

Interview Questions & Answers

Q1: What is a range type and what problem does it solve?

A: A range type represents a continuous range of values (e.g., date intervals, numeric ranges). It solves the "overlap detection" problem: the EXCLUDE USING GIST constraint prevents conflicting bookings/reservations automatically. Without range types, you need complex overlap queries with multiple date comparisons and risk race conditions.

Q2: How does the EXCLUDE constraint differ from UNIQUE?

A: UNIQUE checks for exact equality. EXCLUDE uses a custom operator — typically && (overlap) for ranges. EXCLUDE says "no two rows can have values where this operator returns TRUE". So EXCLUDE USING GIST (room WITH =, period WITH &&) means "no two rows can have the same room AND overlapping periods".

Q3: What is the difference between inet and cidr ?

A: Both store IP addresses, but cidr requires the host bits to be zero — it stores network addresses only (e.g., 192.168.1.0/24). inet allows any host+subnet combination (e.g., 192.168.1.5/24). Use cidr for network prefixes/ACLs; use inet for host addresses.

Q4: What is a tsvector and how is it different from storing text directly?

A: A tsvector is a pre-processed list of lexemes (normalized word stems) with positions and weights. Stopwords are removed, words are stemmed (running → run). Storing text directly and calling to_tsvector() at query time works but is slow for large tables. A GIN-indexed tsvector generated column makes full-text search orders of magnitude faster.

Q5: What does ts_rank measure?

A: ts_rank measures how many times the query terms appear in the document, weighted by position (words near the start rank higher with some configs). ts_rank_cd (cover density) additionally considers how close query terms are to each other. Higher rank = more relevant document.

Q6: How do you implement phrase search in PostgreSQL full-text search?

A: Use the `<->` operator in tsquery: `to_tsquery('quick <-> fox')` matches documents where 'quick' is immediately followed by 'fox'. `<N>` allows a gap of N-1 words. Alternatively, use `phraseto_tsquery('quick brown fox')` which automatically adds proximity operators.

Q7: What index type is used for range types?

A: GIST (Generalized Search Tree). It supports the range operators `&&`, `@>`, `<@`, `<<`, `>>`, `-|-`. GIN indexes do not support range types.

Q8: When would you use PostgreSQL's built-in geometric types vs PostGIS?

A: Built-in geometric types for simple 2D Euclidean geometry: distance calculations, point-in-box checks, overlap detection. PostGIS for any real geospatial work: geodetic coordinates (longitude/latitude on the Earth's surface), spatial reference systems, GIS operations (buffer, intersection, union), WKT/WKB format compatibility, map projections.

Exercises with Solutions

Exercise 1

Create a conference room booking system that prevents overlapping bookings, and write a query to find all available rooms for a given time slot.

Solution:

```
CREATE TABLE rooms (
    room_id    SERIAL PRIMARY KEY,
    name       TEXT    NOT NULL,
    capacity   INTEGER NOT NULL,
    floor      SMALLINT NOT NULL
);

CREATE TABLE bookings (
    booking_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    room_id    INTEGER NOT NULL REFERENCES rooms(room_id),
    title      TEXT    NOT NULL,
    organizer  TEXT    NOT NULL,
    period     TSTZRANGE NOT NULL,
    CONSTRAINT no_overlap EXCLUDE USING GIST (room_id WITH =, period WITH &&)
);

CREATE INDEX idx_bookings_period ON bookings USING GIST (period);

-- Find available rooms for a given slot
SELECT r.*
FROM rooms r
WHERE NOT EXISTS (
    SELECT 1 FROM bookings b
    WHERE b.room_id = r.room_id
        AND b.period && tstzrange('2024-07-15 14:00', '2024-07-15 15:00', '[')')
)
```

```
AND r.capacity >= 10 -- minimum capacity
ORDER BY r.capacity;
```

Exercise 2

Build a full-text search for a knowledge base with title (high weight) and body (lower weight), returning results with highlighted excerpts.

Solution:

```
CREATE TABLE kb_articles (
  id          SERIAL PRIMARY KEY,
  title       TEXT NOT NULL,
  body        TEXT NOT NULL,
  category    TEXT NOT NULL,
  search_vec  TSVECTOR GENERATED ALWAYS AS (
    setweight(to_tsvector('english', title), 'A') ||
    setweight(to_tsvector('english', coalesce(body, '')), 'B')
  ) STORED
);

CREATE INDEX idx_kb_gin ON kb_articles USING GIN (search_vec);

-- Search with ranking and highlights
SELECT
  id, title, category,
  ts_rank_cd(search_vec, q) AS relevance,
  ts_headline('english', body, q,
    'MaxFragments=2, MaxWords=20, MinWords=5, StartSel=**, StopSel=**'
  ) AS excerpt
FROM kb_articles,
     websearch_to_tsquery('english', 'postgresql index performance') AS q
WHERE search_vec @@ q
ORDER BY relevance DESC
LIMIT 10;
```

Cross-References

- [05_json_jsonb.md](#) — JSONB as an alternative to XML
- [06_array_types.md](#) — arrays used in range operations
- [08_constraints.md](#) — EXCLUSION constraints
- [../07_Indexes/](#) — GIST and GIN index internals
- [../11_Advanced_PostgreSQL/](#) — PostGIS and geospatial